Data Structures

BS (CS) _Fall_2025

Lab_13 Task

Learning Objectives:

1. Hashing and Hash Tables

Lab Task 1

Scenario:

You are hired as a **Software Engineer** at **ABC Systems**, a company developing a high-performance password recovery tool. Your task is to design a **Hash Table system** that efficiently stores and retrieves user credentials.

Each user record contains:

- username (string)
- userID (integer)

The system must ensure that:

- Usernames are stored and retrieved efficiently.
- Collisions are handled effectively (using different techniques).

Your manager wants you to **experiment with four different collision handling strategies** to decide which one is best for their product.

Task Description:

Implement **four versions** of a hash table in C++ that store (userID, username) pairs.

Important Notes:

- Each version should be implemented as a **separate class**.
- Maintain and return the **total number of collisions** that occurred during all insertions.
- The **table size** will always be provided dynamically by the test cases.
- Make sure to **print the contents** of your hash table after insertion to verify correctness.

Version 1: Basic Hash Table (No Collision Handling)
Implement a simple hash table without any collision resolution mechanism.

Hash Function:

```
int hashFunction(int key) {
    return key % tableSize;
}
```

Behavior:

- If a collision occurs (i.e., the slot is already occupied), **the new element should not be inserted**.
- Maintain a counter to track how many collisions occurred during all insertions.

Required Functions:

```
void insert(int key, string value);
string search(int key);
void remove(int key);
void display();
int getCollisionCount();
```

Note: The purpose of this task is to test multiple sizes for the Hash Table (using Test Cases) and determine which size should be considered appropriate for designing a Hash Table. In one line as a comment provide the size that you think is best for creating a hash table for your use case along with the reason for choosing the size.

Version 2: Hash Table using Linked List (Chaining)

Objective:
Enhance the basic hash table by using **linked list chaining** to handle collisions.

Hash Function:
Same as above:

```
int hashFunction(int key) {
    return key % tableSize;
}
```

Behavior:

- Each index of the hash table should store a **linked list**.
- If a collision occurs, insert the new node **at the end of the linked list** at that index.
- Increment the collision counter whenever a collision occurs.

Required Functions:

```
void insert(int key, string value);
string search(int key);
void remove(int key);
```

```
void display();
int getCollisionCount();
```

Version 3: Hash Table using Linear Probing

Objective:
Implement open addressing using **linear probing** to handle collisions.

Hash Function:

```
int hashFunction(int key) {
    return key % tableSize;
}
```

Collision Resolution:
When a collision occurs, probe linearly:

```
index = (hash + i) % tableSize;
```

Behavior:

- Increment the collision counter each time probing is needed.
- Continue probing until an empty slot is found or the table is full.

Required Functions:

```
void insert(int key, string value);
string search(int key);
void remove(int key);
void display();
int getCollisionCount();
```

Version 4: Hash Table using Double Hashing

Objective:
Implement open addressing using **double hashing** for improved collision resolution.

Hash Functions:

```
int hash1(int key) { return key % tableSize;}
int hash2(int key) { return 7 - (key % 7); } // secondary hash
```

Collision Resolution:
When a collision occurs, probe using:

```
index = (hash1(key) + i * hash2(key)) % tableSize;
```

Behavior:

- Increment the collision counter for each collision encountered.
- Continue probing until an empty slot is found or the table is full.

Required Functions:

```
void insert(int key, string value);
string search(int key);
void remove(int key);
void display();
int getCollisionCount();
```

Google Test Cases:

```
// BASIC HASH TABLE (NO COLLISION HANDLING)
TEST(BasicHashTableTest, InsertAndCountCollisions_Size6) {
    BasicHashTable table(6);
    table.insert(1, "Alice");
    table.insert(7, "Bob");   // collision -> not inserted
    table.insert(13, "Charlie"); // collision -> not inserted

    EXPECT_EQ(table.getCollisionCount(), 2);
    EXPECT_EQ(table.search(1), "Alice");
    EXPECT_EQ(table.search(7), "");
    EXPECT_EQ(table.search(13), "");

    table.display();
    cout << "Total Collisions: " << table.getCollisionCount() << endl;
}

TEST(BasicHashTableTest, InsertAndCountCollisions_Size7) {
    BasicHashTable table(7);
    table.insert(2, "Tom");
    table.insert(9, "Jerry");  // collision -> not inserted
    table.insert(3, "Sam");
    table.insert(17, "Bob");   // collision -> not inserted

    EXPECT_EQ(table.getCollisionCount(), 2);
```

```
    table.display();
    cout << "Total Collisions: " << table.getCollisionCount() << endl;
}

TEST(BasicHashTableTest, InsertAndCountCollisions_Size8) {
    BasicHashTable table(8);
    table.insert(5, "Eva");
    table.insert(13, "Eve");  // collision -> not inserted
    table.insert(9, "Ray");

    EXPECT_EQ(table.getCollisionCount(), 1);
    EXPECT_EQ(table.search(9), "Ray");

    table.display();
    cout << "Total Collisions: " << table.getCollisionCount() << endl;
}

TEST(BasicHashTableTest, RemoveExistingKey_Size6) {
    BasicHashTable table(6);
    table.insert(4, "Sam");
    table.insert(10, "Ben");  // collision -> not inserted
    table.remove(4);

    EXPECT_EQ(table.search(4), "");
    EXPECT_EQ(table.getCollisionCount(), 1);
    table.display();
    cout << "Total Collisions: " << table.getCollisionCount() << endl;
}

TEST(BasicHashTableTest, SearchNonExistingKey_Size8) {
    BasicHashTable table(8);
    table.insert(1, "Alpha");
    table.insert(9, "Beta");

    EXPECT_EQ(table.search(15), "");
    EXPECT_EQ(table.search(1), "Alpha");
    EXPECT_EQ(table.getCollisionCount(), 1);
    table.display();
    cout << "Total Collisions: " << table.getCollisionCount() << endl;
}

const int TABLE_SIZE = 0; // Manually set the value of your Hash Table size

// HASH TABLE USING LINKED LIST (CHAINING)
```

```
TEST(ChainingHashTableTest, InsertSearchRemoveAndDisplay) {
    ChainingHashTable table(TABLE_SIZE);

    table.insert(1, "Alice");
    table.insert(8, "Bob");
    table.insert(15, "Carol");
    table.insert(3, "Dave");
    table.insert(10, "Eve");

    EXPECT_EQ(table.search(1), "Alice");
    EXPECT_EQ(table.search(8), "Bob");
    EXPECT_EQ(table.search(15), "Carol");
    EXPECT_EQ(table.search(99), "");
    table.remove(8);
    EXPECT_EQ(table.search(8), "");

    table.display();
    cout << "Total Collisions (Chaining): " << table.getCollisionCount() << endl;
}

TEST(ChainingHashTableTest, InsertMultipleSameIndexKeys) {
    ChainingHashTable table(TABLE_SIZE);

    table.insert(2, "A");
    table.insert(9, "B");
    table.insert(16, "C");
    EXPECT_EQ(table.search(16), "C");

    table.display();
    cout << "Total Collisions (Chaining): " << table.getCollisionCount() << endl;
}

TEST(ChainingHashTableTest, RemoveHeadAndTailInChain) {
    ChainingHashTable table(TABLE_SIZE);
    table.insert(5, "X");
    table.insert(12, "Y");
    table.insert(19, "Z");
    table.remove(5);
    table.remove(19);

    EXPECT_EQ(table.search(5), "");
    EXPECT_EQ(table.search(19), "");
    table.display();
    cout << "Total Collisions (Chaining): " << table.getCollisionCount() << endl;
}
```

```
// HASH TABLE USING LINEAR PROBING
TEST(LinearProbingHashTableTest, InsertSearchRemoveAndDisplay) {
    LinearProbingHashTable table(TABLE_SIZE);

    table.insert(1, "John");
    table.insert(8, "Mary");
    table.insert(15, "Alex");
    table.insert(22, "Steve");
    table.insert(3, "Bob");

    EXPECT_EQ(table.search(1), "John");
    EXPECT_EQ(table.search(8), "Mary");
    EXPECT_EQ(table.search(22), "Steve");
    EXPECT_EQ(table.search(99), "");
    table.remove(15);
    EXPECT_EQ(table.search(15), "");

    table.display();
    cout << "Total Collisions (Linear Probing): " << table.getCollisionCount() <<
endl;
}

TEST(LinearProbingHashTableTest, ReinsertAfterRemoval) {
    LinearProbingHashTable table(TABLE_SIZE);
    table.insert(2, "Adam");
    table.insert(9, "Bella");
    table.remove(2);
    table.insert(16, "Chris");

    EXPECT_EQ(table.search(16), "Chris");
    table.display();
    cout << "Total Collisions (Linear Probing): " << table.getCollisionCount() <<
endl;
}

TEST(LinearProbingHashTableTest, InsertFullAndHandleOverflow) {
    LinearProbingHashTable table(5);
    table.insert(1, "A");
    table.insert(2, "B");
    table.insert(3, "C");
    table.insert(4, "D");
    table.insert(5, "E");
    table.insert(6, "F");
```

```
    EXPECT_GE(table.getCollisionCount(), 1);
    table.display();
    cout << "Total Collisions (Linear Probing): " << table.getCollisionCount() <<
endl;
}

// HASH TABLE USING DOUBLE HASHING
TEST(DoubleHashingHashTableTest, InsertSearchRemoveAndDisplay) {
    DoubleHashingHashTable table(TABLE_SIZE);

    table.insert(5, "Liam");
    table.insert(12, "Olivia");
    table.insert(19, "Noah");
    table.insert(26, "Emma");
    table.insert(3, "Ava");

    EXPECT_EQ(table.search(5), "Liam");
    EXPECT_EQ(table.search(12), "Olivia");
    EXPECT_EQ(table.search(26), "Emma");
    EXPECT_EQ(table.search(100), "");
    table.remove(12);
    EXPECT_EQ(table.search(12), "");

    table.display();
    cout << "Total Collisions (Double Hashing): " << table.getCollisionCount() <<
endl;
}

TEST(DoubleHashingHashTableTest, MultipleProbesAndReinsert) {
    DoubleHashingHashTable table(TABLE_SIZE);
    table.insert(2, "Alan");
    table.insert(9, "Beth");
    table.insert(16, "Carl");
    table.remove(9);
    table.insert(23, "Diana");

    EXPECT_EQ(table.search(23), "Diana");
    table.display();
    cout << "Total Collisions (Double Hashing): " << table.getCollisionCount() <<
endl;
}

TEST(DoubleHashingHashTableTest, SearchNonExistingAndCount) {
    DoubleHashingHashTable table(TABLE_SIZE);
    table.insert(4, "Leo");
```

```
    table.insert(11, "Mia");
    EXPECT_EQ(table.search(99), "");
    table.display();
    cout << "Total Collisions (Double Hashing): " << table.getCollisionCount() <<
endl;
}
```